

STEP Cleaning Project Guide

This guide explains how to read this project if you are new to CAD, machine learning, or this codebase.

The project turns a STEP/STP CAD file into face-level geometry and cleaning features, then uses a LightGBM binary classifier to estimate whether each face will stay dirty.

1. The Core Idea

Think of every CAD face as one row in a spreadsheet.

Each row has values such as:

- how exposed the face is
- how much water reaches it
- how much cleaning dose it receives
- whether it is hidden or poorly drained
- whether nearby faces also look difficult to clean

The model learns patterns from previous simulations and then applies those patterns to a new STEP file.

The question the model answers is:

Will this face stay dirty?

2. Recommended Reading Order

Read the project in this order:

1. README.md
2. train_hotspot_classifier.py
3. stepclean/ml/classifier.py
4. stepclean/ml/training_data.py
5. stepclean/prediction/hotspots.py
6. features/cleaning_simulation/
7. features/edge_classification/
8. features/hole_finding/
9. tools/data/cleaning_retention.py
10. tools/diagnostics/hotspot_features.py
11. desktop_ui.py
12. stepclean/app/desktop.py

The root files are launchers. They are intentionally small. The real code is inside folders.

3. Folder Map

<code>train_hotspot_classifier.py</code>	starts model training
<code>analyze_cleaning_particle_retention.py</code>	generates training CSVs
<code>analyze_hotspot_feature_diagnostics.py</code>	analyzes feature usefulness
<code>stepclean/</code>	
<code>ml/</code>	training code
<code>prediction/</code>	prediction code for new STEP files
<code>app/</code>	desktop interface
<code>features/</code>	
<code>cleaning_simulation/</code>	surface sampling and cleaning physics
<code>edge_classification/</code>	convex, concave, neutral edge detection
<code>hole_finding/</code>	face form and inward-cylinder detection
<code>tools/</code>	
<code>data/</code>	data generation tools
<code>diagnostics/</code>	PCA, permutation, and ablation tools
<code>data/training/cleaning_retention_wf05/</code>	current training CSVs
<code>cache/</code>	generated model and prediction outputs
<code>docs/</code>	project guide files

4. Training Data

The current training data is stored here:

```
data/training/cleaning_retention_wf05/
```

The classifier uses three CSV files:

```
face_particle_retention.csv
dirty_surface_neighbor_context.csv
form_particle_retention.csv
```

face_particle_retention.csv

One row per face. This is the main table. It contains:

- object name
- face id
- face form type
- surface area
- cleaning simulation scores
- retained particle count

dirty_surface_neighbor_context.csv

This describes relationships between faces:

- neighboring faces
- boundary type

- convex or concave edge information
- mesh boundary measurements

form_particle_retention.csv

This contains historical retention information by face form. The current compact production model does not use the form feature directly, but this file remains part of the training dataset and can be used in experiments.

5. What The Model Predicts

The production model is a binary classifier.

The target is:

```
stayed_dirty = retained_particle_marker_count > 0
```

If stayed_dirty is 1, that face had at least one retained particle after cleaning.

If stayed_dirty is 0, that face had no retained particles.

The default decision threshold is:

```
dirty_probability >= 0.55
```

6. Production Features

The feature list lives in:

```
stepclean/ml/training_data.py
```

The same feature list is mirrored for inference in:

```
stepclean/prediction/hotspots.py
```

There are 10 production features. They were selected from the strongest SHAP factors in the full model.

Face-Level Cleaning And Sampling

- area_weighted_exposure
- sample_count
- area_weighted_cleaning_dose
- area_weighted_water_dose
- area_weighted_poor_drainage
- area_weighted_hiddeness

These describe whether the face is exposed to cleaning, how much water and cleaning force reached it, how hidden it is, and whether water or dust is likely to remain.

Neighbor Context

- neighbor_area_weighted_cleaning_dose_mean
- neighbor_area_weighted_hotspot_score_max
- neighbor_area_weighted_cleaning_dose_min
- neighbor_area_weighted_hotspot_score_mean

These describe whether nearby faces also look hard to clean. A face can be affected by the cleaning conditions around it, not only by its own geometry.

7. How Training Happens

Training starts here:

```
train_hotspot_classifier.py
```

That launcher calls:

```
stepclean/ml/classifier.py
```

The high-level process is:

1. Read CSVs from data/training/cleaning_retention_wf05/.
2. Build one training row per CAD face.
3. Convert retained_particle_marker_count into stayed_dirty.
4. Clean numeric columns.
5. Keep only the 10 production features.
6. Split by STEP object, not by random face.
7. Train LightGBM.
8. Evaluate accuracy, precision, recall, F1, ROC AUC, and PR AUC.
9. Save the model to cache/lightgbm_hotspot_model.txt.

The grouped split is important. Faces from the same STEP object are kept together. This avoids training on some faces of an object and testing on other faces of the same object.

Current production results:

```
5-fold grouped CV accuracy: 0.8977
Grouped holdout accuracy:   0.8866
Default threshold:          0.55
```

8. How Prediction Works

Prediction code is here:

```
stepclean/prediction/hotspots.py
```

When a new STEP file is evaluated:

1. Read the STEP file.
2. Convert CAD faces into a surface mesh.
3. Run the same style of cleaning simulation used during training.
4. Detect face forms.
5. Classify edges as convex, concave, or neutral.
6. Build the same 10 features for every face.
7. Load `cache/lightgbm_hotspot_model.txt`.
8. Ask LightGBM for a dirty probability.
9. Sort faces by probability.
10. Mark faces as dirty when probability ≥ 0.55 .

The important design rule is:

Training features and prediction features must match.

If a feature is added or removed in training, it must also be added or removed in prediction.

9. Data Generation

The tool that generates training CSVs is:

`tools/data/cleaning_retention.py`

The root launcher is:

`analyze_cleaning_particle_retention.py`

This tool runs cleaning simulations over many STEP files and writes CSVs. Those CSVs are then used by the trainer.

You do not need to run this tool every time. You only need it when rebuilding or extending the training dataset.

10. Diagnostics

The diagnostic tool is:

`tools/diagnostics/hotspot_features.py`

The root launcher is:

`analyze_hotspot_feature_diagnostics.py`

It helps answer questions like:

- which features are important
- which features are redundant
- whether removing a feature improves accuracy
- how much PCA explains
- how much prediction quality drops when a feature is permuted

This is analysis code. It is not part of normal prediction.

11. Generated Outputs

The cache/ folder is for generated outputs.

Current expected files:

```
cache/lightgbm_hotspot_model.txt  
cache/hotspot_dirty_classifier_predictions.csv  
cache/hotspot_dirty_classifier_holdout_predictions.csv  
cache/hotspot_dirty_classifier_feature_importance.csv
```

The model file is used during prediction. The other files are useful for inspection.

12. Environments

The project uses two Python environments because CAD tooling and ML tooling have different dependency needs.

ML Environment

Use this for training and diagnostics:

```
.\.venv311\Scripts\python.exe train_hotspot_classifier.py  
.\.venv311\Scripts\python.exe analyze_hotspot_feature_diagnostics.py
```

CAD Environment

Use this for CAD-dependent code and tests:

```
.\occw\env\python.exe -m unittest discover -s tests
```

13. Safe Changes

If you want to change the model:

1. Change feature generation in stepclean/ml/training_data.py.
2. Make the same feature change in stepclean/prediction/hotspots.py.
3. Retrain with train_hotspot_classifier.py.
4. Check CV and holdout accuracy.
5. Verify prediction still works.

If you change cleaning physics, inspect:

```
features/cleaning_simulation/
```

If you change CAD shape recognition, inspect:

```
features/edge_classification/  
features/hole_finding/
```

14. Mental Model

The model learns patterns like:

```
faces with low exposure, poor drainage, high hiddenness, or hard-to-clean  
neighbors are more likely to stay dirty
```

The prediction code repeats the same feature-building process for a new STEP file and asks:

```
Which faces look like the dirty faces from training?
```

That is the core idea of the whole project.

15. Desktop UI Interface

The desktop interface is intentionally last in this guide because it is mostly a viewer around the model and prediction logic.

Start it with:

```
.\occwl-env\python.exe desktop_ui.py
```

Execution path:

```
desktop_ui.py  
-> imports main from stepclean.app.desktop  
-> stepclean.app.desktop.main()  
-> opens the Tkinter and OpenCascade viewer
```

The main UI implementation is:

```
stepclean/app/desktop.py
```

Important controls:

- Open STEP File: loads a STEP/STP file.
- Predict Dirty Faces: runs the trained classifier and colors predicted dirty faces in pink.
- Run Cleaning Simulation: runs the cleaning simulation and colors retained-particle faces.
- Compare Model vs Simulation: shows classifier prediction together with simulated retained-particle faces.
- Water Force: sets cleaning force from 0.000 to 1.000; default is 0.500.
- Dirty Threshold: colors model-predicted dirty faces whose probability is at least this value; default is 0.550.
- Spray Direction: chooses all-axis spray or one side.

If you only change UI labels, colors, buttons, or layout, edit:

```
stepclean/app/desktop.py
```